

Guia Didático: Tipos de Classes em Java

Público-alvo

Estudantes do curso de Desenvolvimento de Sistemas.

Objetivos

- Identificar os principais tipos de classes em Java.
 - Entender o conceito e a finalidade de cada tipo.
 - Ler, executar e modificar exemplos práticos em Java.
 - Relacionar código-fonte com diagramas UML.
 - Resolver exercícios e conferir soluções completas.
-

1. Introdução

Em Java, uma **classe** é um molde usado para criar objetos. Ela pode conter atributos, métodos, construtores e diferentes regras de uso. Existem diversos tipos de classes, cada um adequado para uma necessidade específica do projeto.

Neste material, estudaremos os seguintes tipos de classes:

1. Classe concreta
 2. Classe abstrata
 3. Classe final
 4. Classe interna (inner class)
 5. Classe aninhada estática (static nested class)
 6. Classe local
 7. Classe anônima
 8. Enum class
 9. Record class
 10. Classe sealed
-

2. Tipos de Classes

2.1 Classe concreta

Conceito

É a classe comum, que pode ser instanciada normalmente com new. Ela possui implementação completa e serve para representar objetos do sistema.

UML

```
classDiagram
class Pessoa {
    -nome : String
    -idade : int
    +Pessoa(nome:String, idade:int)
    +apresentar() : void
}
```

Código Java

// Arquivo: PessoaDemo.java

// Classe concreta: pode ser instanciada diretamente.

```
class Pessoa {
    // Atributos da classe.
    private String nome;
    private int idade;

    // Construtor para inicializar os atributos.
    public Pessoa(String nome, int idade) {
        this.nome = nome;
        this.idade = idade;
    }

    // Método que exibe os dados do objeto.
    public void apresentar() {
        System.out.println("Nome: " + nome + ", Idade: " + idade);
    }
}

// Classe principal com método main.
public class PessoaDemo {
    public static void main(String[] args) {
        // Criando um objeto da classe concreta.
        Pessoa p = new Pessoa("Ana", 17);
        p.apresentar();
    }
}
```

2.2 Classe abstrata

Conceito

É uma classe que **não pode ser instanciada diretamente**. Ela é usada como base para outras classes. Pode conter métodos concretos e métodos abstratos.

UML

```
classDiagram
class Funcionario {
    <<abstract>>
    -nome : String
    +Funcionario(nome:String)
    +mostrarNome() : void
    +calcularSalario() : double
}
class FuncionarioCLT {
    -salarioBase : double
    +FuncionarioCLT(nome:String, salarioBase:double)
    +calcularSalario() : double
}
Funcionario <|-- FuncionarioCLT
```

Código Java

// Arquivo: FuncionarioDemo.java

// Classe abstrata: serve de modelo para subclasses.

```
abstract class Funcionario {
    // Atributo protegido para ser reutilizado nas subclasses.
    protected String nome;

    // Construtor da classe abstrata.
    public Funcionario(String nome) {
        this.nome = nome;
    }

    // Método concreto: já possui implementação.
    public void mostrarNome() {
        System.out.println("Funcionário: " + nome);
    }

    // Método abstrato: cada subclasse deve implementar.
    public abstract double calcularSalario();
}
```

// Subclasse concreta que implementa o método abstrato.

```

class FuncionarioCLT extends Funcionario {
    private double salarioBase;

    public FuncionarioCLT(String nome, double salarioBase) {
        super(nome);
        this.salarioBase = salarioBase;
    }

    @Override
    public double calcularSalario() {
        return salarioBase;
    }
}

public class FuncionarioDemo {
    public static void main(String[] args) {
        // Não é possível fazer: new Funcionario(...)
        Funcionario f = new FuncionarioCLT("Carlos", 3500.0);
        f.mostrarNome();
        System.out.println("Salário: R$ " + f.calcularSalario());
    }
}

```

2.3 Classe final

Conceito

É uma classe que **não pode ser herdada**. Serve para impedir que outras classes alterem seu comportamento por herança.

UML

```

classDiagram
class UtilitarioMatematica {
    <<final>>
    +somar(a:int, b:int) : int
}

```

Código Java

```

// Arquivo: FinalDemo.java

// Classe final: não pode ser estendida.
final class UtilitarioMatematica {
    // Método simples de soma.
    public int somar(int a, int b) {
        return a + b;
    }
}

```

```

public class FinalDemo {
    public static void main(String[] args) {
        // Criando objeto normalmente.
        UtilitarioMatematica u = new UtilitarioMatematica();
        System.out.println("Soma: " + u.somar(10, 5));
    }
}

```

2.4 Classe interna (inner class)

Conceito

É uma classe declarada dentro de outra classe, sem o modificador `static`. Ela está ligada a um objeto da classe externa.

UML

```

classDiagram
class Computador {
    -marca : String
    +Computador(marca:String)
}
class Processador {
    -modelo : String
    +Processador(modelo:String)
    +mostrarDados() : void
}
Computador *-- Processador

```

Código Java

// Arquivo: InnerClassDemo.java

```

class Computador {
    // Atributo da classe externa.
    private String marca;

    public Computador(String marca) {
        this.marca = marca;
    }

    // Classe interna: depende de um objeto Computador.
    class Processador {
        private String modelo;

        public Processador(String modelo) {
            this.modelo = modelo;
        }
    }
}

```

```

    }

    public void mostrarDados() {
        // A classe interna pode acessar atributos da classe externa.
        System.out.println("Marca do computador: " + marca);
        System.out.println("Modelo do processador: " + modelo);
    }
}

}

}

public class InnerClassDemo {
    public static void main(String[] args) {
        // Criando objeto externo.
        Computador pc = new Computador("Dell");

        // Criando objeto da classe interna associado ao objeto externo.
        Computador.Processador proc = pc.new Processador("Intel i5");
        proc.mostrarDados();
    }
}

```

2.5 Classe aninhada estática (static nested class)

Conceito

É uma classe declarada dentro de outra classe com a palavra-chave `static`. Ela não depende de um objeto da classe externa.

UML

```

classDiagram
class Empresa {
    +nome : String
}
class Departamento {
    -nome : String
    +Departamento(nome:String)
    +mostrar() : void
}
Empresa +-- Departamento

```

Código Java

// Arquivo: *StaticNestedDemo.java*

```

class Empresa {
    // Classe aninhada estática: não depende de instância da classe
    externa.
    static class Departamento {

```

```

        private String nome;

        public Departamento(String nome) {
            this.nome = nome;
        }

        public void mostrar() {
            System.out.println("Departamento: " + nome);
        }
    }
}

public class StaticNestedDemo {
    public static void main(String[] args) {
        // A classe aninhada estática pode ser criada sem objeto externo.
        Empresa.Departamento d = new Empresa.Departamento("Financeiro");
        d.mostrar();
    }
}

```

2.6 Classe local

Conceito

É uma classe declarada dentro de um método. Seu uso é restrito ao bloco em que foi criada.

UML

```

classDiagram
class Relatorio {
    +gerar() : void
}
class Cabecalho {
    -titulo : String
    +Cabecalho(titulo:String)
    +mostrar() : void
}
Relatorio ..> Cabecalho

```

Código Java

// Arquivo: LocalClassDemo.java

```

class Relatorio {
    public void gerar() {
        // Classe local: só existe dentro deste método.
        class Cabecalho {
            private String titulo;

```

```

        public Cabecalho(String titulo) {
            this.titulo = titulo;
        }

        public void mostrar() {
            System.out.println("=== " + titulo + " ===");
        }
    }

    // Criando e usando a classe Local.
    Cabecalho cab = new Cabecalho("Relatório de Vendas");
    cab.mostrar();
    System.out.println("Conteúdo do relatório...");
}

public class LocalClassDemo {
    public static void main(String[] args) {
        Relatorio r = new Relatorio();
        r.gerar();
    }
}

```

2.7 Classe anônima

Conceito

É uma classe sem nome, criada geralmente para implementar rapidamente uma interface ou sobrescrever um método de uma classe.

UML

```

classDiagram
class SaudacaoApp {
    +main(args:String[]) : void
}
interface Saudacao {
    +mostrarMensagem() : void
}
SaudacaoApp ..> Saudacao

```

Código Java

```

// Arquivo: AnonimaDemo.java

// Interface usada no exemplo.
interface Saudacao {

```



```

        void mostrarMensagem();
    }

    public class AnonimaDemo {
        public static void main(String[] args) {
            // Classe anônima implementando a interface Saudacao.
            Saudacao s = new Saudacao() {
                @Override
                public void mostrarMensagem() {
                    System.out.println("Olá! Esta mensagem vem de uma classe
anônima.");
                }
            };

            // Chamando o método da classe anônima.
            s.mostrarMensagem();
        }
    }
}

```

2.8 Enum class

Conceito

Uma enum class representa um conjunto fixo de constantes. É útil para dias da semana, estados, níveis e opções limitadas.

UML

```

classDiagram
class StatusPedido {
    <<enumeration>>
    PENDENTE
    PROCESSANDO
    ENVIADO
    ENTREGUE
}

```

Código Java

// Arquivo: EnumDemo.java

// Enum class: conjunto fixo de constantes.

```

enum StatusPedido {
    PENDENTE,
    PROCESSANDO,
    ENVIADO,
    ENTREGUE
}

```

```

public class EnumDemo {
    public static void main(String[] args) {
        // Variável do tipo enum.
        StatusPedido status = StatusPedido.PROCESSANDO;

        // Exibindo o valor escolhido.
        System.out.println("Status atual: " + status);
    }
}

```

2.9 Record class

Conceito

record é um tipo especial de classe usado para representar dados de forma mais simples. O Java gera automaticamente construtor, getters, toString, equals e hashCode.

UML

```

classDiagram
class Aluno {
    <<record>>
    +nome : String
    +idade : int
}

```

Código Java

// Arquivo: RecordDemo.java

```

// Record class: criada para representar dados de forma compacta.
record Aluno(String nome, int idade) {
    // Bloco opcional para validações.
    public Aluno {
        if (idade < 0) {
            throw new IllegalArgumentException("Idade não pode ser
negativa.");
        }
    }
}

```

```

public class RecordDemo {
    public static void main(String[] args) {
        // Criando um objeto record.
        Aluno a = new Aluno("Marina", 18);

        // Métodos nome() e idade() são gerados automaticamente.
    }
}

```

```

        System.out.println("Nome: " + a.nome());
        System.out.println("Idade: " + a.idade());
        System.out.println(a);
    }
}

```

2.10 Classe sealed

Conceito

Uma classe sealed restringe quais classes podem herdá-la. Isso aumenta o controle sobre a hierarquia.

UML

```

classDiagram
class Veiculo {
    <<sealed>>
}
class Carro {
    <<final>>
}
class Moto {
    <<final>>
}
Veiculo <|-- Carro
Veiculo <|-- Moto

```

Código Java

// Arquivo: SealedDemo.java

// Classe sealed: restringe explicitamente as subclasses permitidas.

```

sealed class Veiculo permits Carro, Moto {
    public void mover() {
        System.out.println("O veículo está em movimento.");
    }
}

```

// Classe final permitida pela classe sealed.

```

final class Carro extends Veiculo {
    public void abrirPortaMalas() {
        System.out.println("Porta-malas aberto.");
    }
}

```

// Outra classe final permitida.

```

final class Moto extends Veiculo {
    public void empinar() {

```

```

        System.out.println("A moto empinou com segurança no exemplo
didático.");
    }
}

public class SealedDemo {
    public static void main(String[] args) {
        Carro c = new Carro();
        c.mover();
        c.abrirPortaMalas();

        Moto m = new Moto();
        m.mover();
        m.empinar();
    }
}

```

3. Tabela-resumo dos tipos de classes

Tipo de classe	Pode instanciar?	Pode herdar?	Finalidade principal	Exemplo de uso
Concreta	Sim	Sim, se não for final	Representar objetos comuns	Pessoa, Produto
Abstrata	Não	Sim	Servir de base para subclasses	Funcionario
Final	Sim	Não	Impedir herança	Classe utilitária ou sensível
Interna	Sim, com objeto externo	Sim	Organizar melhor elementos ligados à classe externa	Computador.Processador
Aninhada estática	Sim	Sim	Agrupar classes relacionadas sem depender de objeto externo	Empresa.Departamento
Local	Sim, apenas	Em escopo	Uso	Cabecalho

Tipo de classe	Pode instanciar?	Pode herdar?	Finalidade principal	Exemplo de uso
	dentro do método	local	temporário dentro de um método	
Anônima	Sim, no ponto de criação	Não diretamente	Implementação rápida e pontual	Eventos, callbacks
Enum	Sim, como constantes predefinidas	Não como classe comum	Representar conjuntos fixos de valores	StatusPedido
Record	Sim	Não (implicitamente final)	Modelar dados de forma compacta	Aluno
Sealed	Depende da implementação	Apenas classes permitidas	Controlar hierarquias de herança	Veiculo

6. Conclusão

O estudo dos tipos de classes em Java ajuda o aluno a compreender que nem toda classe é usada da mesma maneira. Algumas servem para modelar objetos comuns, outras para definir regras de herança, algumas organizam melhor o código, e outras ajudam a representar dados de forma mais simples.

Para a aprendizagem dos estudantes, vale a pena praticar estes conceitos com pequenas alterações nos exemplos, como: - trocar nomes de classes e atributos; - adicionar novos métodos; - testar herança e polimorfismo; - desenhar os diagramas UML a partir do código; - pedir que os alunos comparem dois tipos de classes e expliquem quando cada um deve ser usado.

7. Sugestões didáticas para uso em sala

- Apresente primeiro **classe concreta**, **abstrata** e **final**, pois formam a base.
- Depois avance para **inner**, **static nested**, **local** e **anônima**, mostrando a diferença de escopo.
- Em seguida, trabalhe **enum**, **record** e **sealed**, relacionando com recursos mais modernos do Java.

- Peça aos alunos para executarem cada exemplo e depois modificarem o código.
- Use os diagramas UML como apoio visual para reforçar o entendimento estrutural.